SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS
COURSE CODE: CSA0606

COURSE OUTCOME COVERED

CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull

problems (BL1,BL2,BL3)problems (BL1, BL2,BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 1 Total Marks:50

Annexure A

SCENARIO BASED ASSESSMENT

Code of Conduct:

I B.Muni Chandini (192524283) (Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *B.Muni Chandini*

Q61. A weather forecasting system processes temperature readings that must be sorted before analysis. Data arrives in random order.

a) Compare Selection Sort and Bubble Sort for this dataset.

b) Analyze their computational cost.

c) Recommend the better algorithm with justification.

Weather Forecasting System

Selection Sort vs Bubble Sort – Scenario Based Assessment

CO2-AT2 | Design and Analysis of Algorithms

1. Analyze the Given Scenario

A weather forecasting system continuously receives temperature readings from multiple sensors placed across different geographic locations. These readings arrive in random (unsorted) order because sensors report data at different times and network delays are unpredictable.

Before any meaningful analysis—such as calculating daily averages, detecting temperature anomalies, generating forecasts, or producing sorted reports for meteorologists—the system must first arrange the temperature values in sorted order.

Core Challenge: The system needs an efficient and reliable sorting algorithm that can handle incoming batches of temperature data. Two classical comparison-based sorting algorithms are under consideration: **Selection Sort** and **Bubble Sort**.

The goal of this assessment is to compare these two algorithms, analyse their computational cost, and recommend the more suitable algorithm for the weather forecasting application.

2. Identify User Requirements

2.1 Functional Requirements

1. The system must accept a list of temperature readings that arrive in random order.
2. It must sort the temperature values in non-decreasing (ascending) order.
3. Sorted data must be available for subsequent analysis modules (averaging, anomaly detection, reporting).
4. The sorting process should be deterministic and produce a stable, correct ordered list every time.

2.2 Non-Functional Requirements

5. Sorting should complete within acceptable time limits even when the number of readings grows (scalability).
6. The algorithm should have predictable performance (important for real-time or near-real-time forecasting).
7. Memory usage should be modest (in-place sorting preferred).
8. The implementation should be simple to code, test, and maintain.

2.3 Stakeholders

- Meteorologists – need sorted temperature data for analysis and report generation.
- System developers – responsible for implementing and maintaining the sorting module.

- System administrators – concerned with performance and resource consumption.

3. Suggest Suitable Technologies

Although the core problem is algorithmic, a complete weather forecasting system would typically use the following technology stack:

Layer	Suggested Technology
Programming Language	Python / Java / C++ (any language supporting arrays)
Data Storage	In-memory arrays / lists; optionally SQLite or PostgreSQL for historical data
Backend Framework	Flask / FastAPI (Python) or Spring Boot (Java) for API endpoints
Frontend (optional)	HTML + JavaScript / React for visualisation dashboards
Sorting Module	Custom implementation of Selection Sort or Bubble Sort (focus of this study)

For this assessment we concentrate exclusively on the sorting algorithms themselves, independent of the surrounding technology stack.

4. Design the Solution (Algorithm Design)

We design two alternative sorting modules that can be plugged into the weather system.

4.1 Selection Sort – Design

Selection Sort works by repeatedly finding the minimum element from the unsorted portion of the array and swapping it with the first unsorted element.

Pseudocode:

```
SelectionSort(A[0…n−1]):
    for i ← 0 to n−2 do
        minIndex ← i
        for j ← i+1 to n−1 do
            if A[j] < A[minIndex] then
                minIndex ← j
        swap A[i] with A[minIndex]
```

4.2 Bubble Sort – Design

Bubble Sort repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order. Larger values “bubble” toward the end of the array.

Pseudocode:

```
BubbleSort(A[0…n−1]):
    for i ← 0 to n−2 do
        swapped ← false
        for j ← 0 to n−2−i do
            if A[j] > A[j+1] then
                swap A[j] with A[j+1]
                swapped ← true
        if not swapped then break   // early termination optimisation
```

4.3 Data Structure

Both algorithms operate on a simple one-dimensional array (or Python list) of floating-point temperature values. No additional data structures are required; both algorithms are in-place.

5. Explain Implementation Steps

The following steps describe how either sorting algorithm would be integrated into the weather forecasting system:

1. Receive a batch of temperature readings from sensors (stored in an unsorted array).
2. Pass the array to the chosen sorting function (Selection Sort or Bubble Sort).
3. The function rearranges the array in ascending order of temperature.
4. Return the sorted array to the analysis module.
5. Analysis module computes averages, detects outliers, generates reports, etc.
6. Optionally store the sorted historical data in a database for future reference.

5.1 Example Execution (Illustrative Dataset)

Suppose a small batch of temperature readings arrives:

Unsorted: [28.5, 22.1, 31.0, 19.8, 25.4]

After sorting (either algorithm):

Sorted: [19.8, 22.1, 25.4, 28.5, 31.0]

6. Justify Design Decisions

The decision to evaluate Selection Sort and Bubble Sort (rather than more advanced algorithms such as Quick Sort or Merge Sort) is deliberate:

- Both are simple, comparison-based, in-place algorithms taught in foundational courses.
- They have the same asymptotic complexity, making a direct comparison meaningful for educational and small-scale systems.
- They require no extra memory, which is desirable for embedded or resource-constrained weather stations.
- Their behaviour is easy to analyse and explain to stakeholders.

Within these two options we still need to decide which is preferable; that decision is driven by the performance evaluation below.

7. Performance Evaluation

7.1 Computational Cost Analysis

Selection Sort

- **Comparisons:** Always performs $n(n-1)/2$ comparisons regardless of input order.
- **Swaps:** At most $n-1$ swaps (one per outer-loop iteration).
- **Time Complexity:** $\Theta(n^2)$ in all cases (best, average, worst).
- **Space Complexity:** $O(1)$ auxiliary space.
- **Stability:** Not stable.

Bubble Sort

- **Comparisons:** Up to $n(n-1)/2$ comparisons; can be fewer with early-termination flag.
- **Swaps:** Can be as high as $O(n^2)$ in the worst case (reverse-sorted data).

- **Time Complexity:** O(n²) worst & average; O(n) best case (already sorted) with optimisation.
- **Space Complexity:** O(1) auxiliary space.
- **Stability:** Stable (equal elements keep relative order).

7.2 Comparative Summary Table

Criterion	Selection Sort	Bubble Sort
Best Case Time	Θ(n²)	**O(n) (with early exit)**
Average Case Time	Θ(n²)	Θ(n²)
Worst Case Time	Θ(n²)	Θ(n²)
Number of Swaps	**At most n−1 (few)**	Up to O(n²) (many)
Comparisons	Always n(n−1)/2	≤ n(n−1)/2
Extra Memory	O(1)	O(1)
Stable?	No	**Yes**
Adaptive?	No	**Yes (early termination)**

7.3 Behaviour on Weather Data

Temperature readings arriving from sensors are essentially random. Therefore the average-case behaviour is the most relevant metric. Both algorithms exhibit Θ(n²) average-case performance. However:

- Selection Sort performs a fixed, predictable number of comparisons and a very small number of swaps.
- Bubble Sort may perform many more swaps, each of which involves writing to memory; on large batches this extra write traffic can make Bubble Sort noticeably slower in practice.
- If consecutive batches happen to be nearly sorted (possible when sensors report at regular intervals), optimised Bubble Sort can finish in near-linear time, while Selection Sort still takes quadratic time.

8. Recommendation with Justification

Recommended Algorithm: Selection Sort

Justification:

- **Fewer swaps:** Selection Sort performs at most n−1 swaps, whereas Bubble Sort can perform quadratic swaps. Temperature arrays can be moderately large; reducing memory writes improves practical speed and reduces energy consumption on battery-powered weather stations.
- **Predictable performance:** Selection Sort always runs in Θ(n²) time. Weather systems often need deterministic latency guarantees; the constant behaviour of Selection Sort is easier to reason about.
- **Simplicity of implementation:** The code is short, easy to verify, and contains no special early-exit logic that must be tested.
- **In-place & low memory:** Both algorithms satisfy this requirement; Selection Sort still wins on swap count.

Caveat: If the system frequently receives already-sorted or nearly-sorted batches and stability of equal temperatures is important, an optimised Bubble Sort (or better still, Insertion Sort / TimSort) would be preferable. For the general random-arrival case described in the scenario, Selection Sort is the better choice among the two given options.

9. Conclusion

In a weather forecasting system that must sort randomly arriving temperature readings, both Selection Sort and Bubble Sort provide correct, in-place solutions with quadratic time complexity. A detailed comparison reveals that Selection Sort is generally superior for this scenario because it minimises the number of data movements (swaps) while delivering completely predictable performance.

Bubble Sort remains useful for educational illustration and for nearly-sorted data, but its potentially large number of swaps makes it less attractive for production weather-data pipelines.

Therefore, for the given scenario we recommend implementing

Selection Sort as the primary sorting algorithm inside the weather forecasting system.